



UNIVERSIDAD
CAECE

Cámara Argentina de Comercio y Servicios

TECNICATURA EN PROGRAMACIÓN

BASES DE DATOS

GUIA DE LA UNIDAD Nro. 3
OBTENCIÓN DE INFORMACIÓN

Contenidista: Lic. Pablo R. Fernández

ÍNDICE – UNIDAD 3:

INTRODUCCIÓN.....	5
1. SENTENCIAS DDL: CREATE, ALTER Y DROP	5
ACTIVIDAD 1	8
2. USO DE SENTENCIAS DML: SELECT, INSERT, UPDATE, DELETE	9
ACTIVIDAD 2	14
3. USO DE SENTENCIAS DML: USO DE SUBCONSULTAS.....	14
ACTIVIDAD 3	18
4. USO DE FUNCIONES AGREGADAS.....	18
ACTIVIDAD 4	20
5. USO DE SENTENCIAS DML PARA MODIFICAR DATOS: INSERT, UPDATE, DELETE.....	21
ACTIVIDAD 5	23
6. INTERACCIÓN CON NULLS.....	24
7. TRANSACCIONES	27
ACTIVIDAD 6	29
SÍNTESIS DE LA UNIDAD	30

MAPA DE LA UNIDAD 3: Lenguaje SQL

PROPÓSITOS

En esta unidad nos proponemos explicarte como crear una base de datos desde un Diagrama Entidad-Relación (o DER por sus siglas en inglés). También como agregarle datos y luego como consultarlos.

OBJETIVOS

- ✓ Analizar el uso de sentencias DDL: create, alter, drop.
- ✓ Analizar el uso de las sentencias DML: select.
- ✓ Conocer cómo funcionan las consultas.
- ✓ Conocer cómo funcionan las funciones agregadas.
- ✓ Conocer cómo funcionan las sentencias para modificar datos.
- ✓ Conocer cómo interactuar con NULLs.
- ✓ Conocer cómo funcionan las transacciones.

CONTENIDOS

Para que alcance los objetivos, los contenidos que abordará son los siguientes:

- 1) Registración y activación de una usuario en <https://sqliteonline.com/>
- 2) Interacción con el entorno <https://sqliteonline.com/>
- 3) Uso de sentencias DDL: create, alter y drop
- 4) Uso de sentencias DML: select, insert, update, delete.
- 5) Uso de sentencias DML: Uso de subconsultas.
- 6) Uso de funciones agregadas.
- 7) Uso de subconsultas.
- 8) Interacción con NULLs.
- 9) Transacciones

PALABRAS CLAVES

DML, DDL, select, insert, update, delete, NULL

**BIBLIOGRAFÍA DE CONSULTA**

ELMASRI, RAMEZ/NAVATHE, SHAMKANT(2011). *Fundamentos de Sistemas de Base de Datos*. 6ta Ed, EEUU:Pearson / Addison Wesley.

INTRODUCCIÓN

Para comenzar con esta unidad, y continuando desde la Unidad 2 donde vimos cómo obteníamos el esquema interno, es decir las tablas de nuestra base de datos, veremos a partir del modelo E-R cómo crear la base de datos y sus tablas. Aprenderemos a partir de los mismos ejemplos que trabajamos en la Unidad 2 de manera de unir todos los conocimientos adquiridos.

Comencemos...



1. SENTENCIAS DDL: CREATE, ALTER Y DROP

A partir de ahora comenzaremos a ver las distintas sentencias del lenguaje SQL (del inglés Structured Query Language) en su forma compatible con ANSI 92. ANSI es un standard del lenguaje, lo que nos asegura que dicha forma es compatible con todos los DBMS del mercado. Es decir que si escribimos una sentencia de código SQL ANSI 92 la podremos ejecutar exitosamente en cualquier DBMS. Si la sentencia no es ANSI, podría darse el caso que se ejecute con éxito en un DBMS (por ejemplo SQL Server) pero no en otros (por ejemplo Oracle y MySQL).



Como vimos en la Unidad 1, el lenguaje DDL o de definición de datos, contiene sentencias que permiten crear, modificar o eliminar objetos en el esquema interno de la base de datos en base al esquema conceptual. Para ello se utilizan tres sentencias muy potentes, ellas son CREATE, ALTER y DROP, las que veremos a continuación.

La sentencia CREATE tiene la siguiente estructura en su forma más simple:

CREATE<tipo de objeto> Nombre de objeto

Luego dependiendo del tipo de objeto la sentencia varía. Como nos vamos a concentrar por ahora en la creación de tablas, vamos a ver en detalle la sentencia para este caso:

CREATE TABLE<Nombre de la tabla> (
Columna1 tipo de dato NULL,
Columna2 tipo de dato NULL,
....
)

Los tipos de datos pueden ser:

Int: números enteros

Datetime: fecha y hora

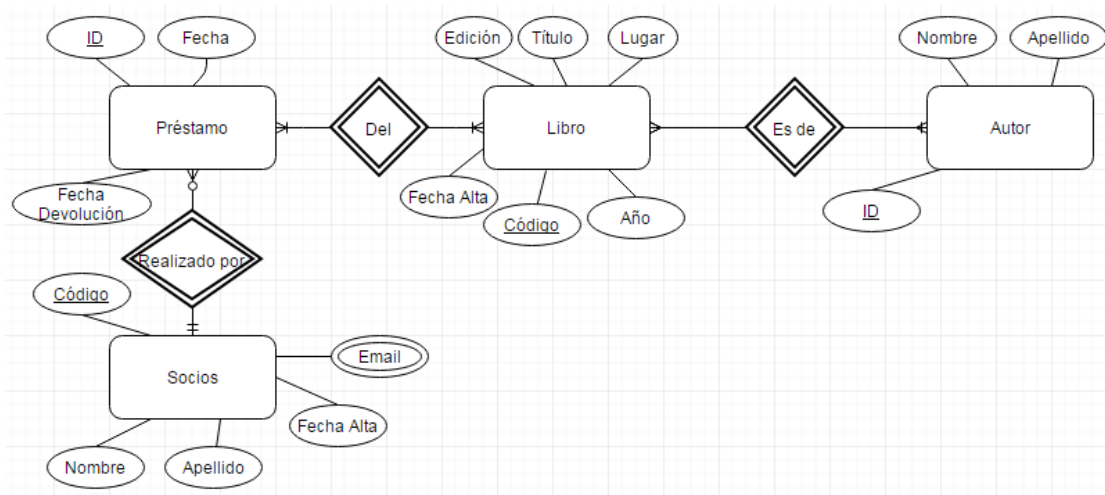
Varchar(longitud en cantidad de caracteres): cadenas de caracteres o strings

Float: números decimales

Son los tipos más comunes que estaremos manejando.



Retomamos de la Unidad 2 el esquema interno que nos había quedado para el modelo de los préstamos de libros de la biblioteca, definido por el diagrama E-R que copiamos aquí:



Para crear la tabla Préstamo (tener en cuenta que no vamos a poner el acento en la e para el nombre de la tabla):

```

CREATE TABLE prestamo (
    ID int not null,
    Fecha datetime null,
    Fecha_Devolucion datetime null,
    PRIMARY KEY (ID),
    FOREIGN KEY (CodSocio) REFERENCES Socios(Codigo))
    
```

Hemos marcado con resaltadores de diferentes colores las palabras reservadas según si son tipos de datos: **púrpura**, sentencias SQL: **celeste**, otros: **verde y gris** para los NULL.

El NULL es el valor que tiene un atributo cuando no se le ha asignado aún ningún valor. Por ejemplo, al momento de crear una tabla, ésta no tiene ninguna t-upla. Luego los usuarios comienzan a agregar datos y puede suceder que no los conozcan todos, por ejemplo que al dar de alta un libro, no tengan a mano el año de impresión. Por lo tanto lo dejan vacío. El DBMS va a tomar este valor no ingresado como NULL.

Cuando creamos una tabla, y ponemos en las columnas NULL quiere decir que ese atributo no es requerido o sea que se podría dejar vacío cuando se ingresan datos. Si ponemos not NULL quiere decir que no permitiremos que esa columna quede en blanco, es decir que será obligatorio para los usuarios llenar ese campo y el DBMS se encargará de indicarle al usuario que debe ingresar un valor.

La cláusula Primary Key contiene entre paréntesis el nombre del atributo que será utilizado como clave de la tabla.



Por último, recordemos que teníamos la clave foránea que correspondía al código del socio que solicitaba el préstamo. Esa clave foránea se crea con la cláusula Foreign Key indicando en "References" el nombre de la tabla y el atributo relacionado.

De la misma manera, si queremos crear la tabla Socios, la sentencia será:

```
CREATE TABLE socios (
Codigo int not null,
Nombre varchar(30) null,
Apellido varchar(30) null,
Fecha_Alta datetime null,
PRIMARY KEY (Codigo))
```

Además tenemos el atributo multivaluado Email que habíamos visto que se pasaba como una tabla. Este se crea de la siguiente manera:

```
CREATE TABLE Email (
Codigo int not null,
Email varchar(100) null,
PRIMARY KEY (Codigo),
FOREIGN KEY (CodSocio) REFERENCES Socios(Codigo))
```



¿Podrás poner a continuación la sentencia de creación de las tablas que faltan?

Si seguiste los pasos indicados entonces habrás creado las tablas restantes de la siguiente manera (recordarás de la Unidad 2 que las relaciones N-M se pasaban como una tabla adicional con los atributos clave de las tablas participantes en la relación, como clave de dicha tabla):

```
CREATE TABLE Libro(
Codigo int not null,
Titulo varchar(100) null,
Edición varchar(20) null,
Lugar varchar(50) null,
Año int null,
Fecha_Alta datetime null,
PRIMARY KEY (Codigo))
```

```
CREATE TABLE Libro_Autor (
```

```
CREATE TABLE Autor(
ID int not null,
Nombre varchar(30) null,
Apellido varchar(30) null,
PRIMARY KEY (ID))
```

```
CREATE TABLE Prestamo_Libro (
```

```
CodLibro int not null,  
CodAutor int not null,  
PRIMARY KEY (CodLibro,CodAutor)  
FOREIGN KEY (CodLibro)  
REFERENCES Libro(Codigo),  
FOREIGN KEY (CodAutor)  
REFERENCES Autor(ID))
```

```
CodLibro int not null,  
CodPrestamo int not null,  
PRIMARY KEY (CodLibro,CodPrestamo),  
FOREIGN KEY (CodLibro)  
REFERENCES Libro(Codigo),  
FOREIGN KEY (CodPrestamo)  
REFERENCES Prestamo(ID))
```



Ahora bien, te estarás preguntando ¿Qué sucede si quisiéramos cambiar algo en la tabla que creamos?, por ejemplo ¿Cómo haríamos si quisiéramos agregar una columna?

En ese caso utilizamos la sentencia ALTER. Para agregar una columna haríamos así:

```
ALTER TABLE nombre de tabla  
ADD nombre de columna tipo de dato;  
O para agregar una columna haríamos así:  
ALTER TABLE nombre de tabla  
DROP COLUMN nombre de columna;
```

Para eliminar un objeto utilizaremos la sentencia DROP indicando el tipo de objeto y su nombre.

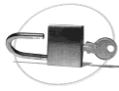
Por ejemplo, para eliminar la tabla socios haríamos lo siguiente:

```
DROP TABLE socios;
```



ACTIVIDAD 1

Le pedimos que a continuación realice la Actividad de autoevaluación que le proponemos en el campus para ir afianzando los conceptos vistos hasta aquí



ABORDEMOS EL LOGRO DEL SEGUNDO OBJETIVO DE LA UNIDAD:

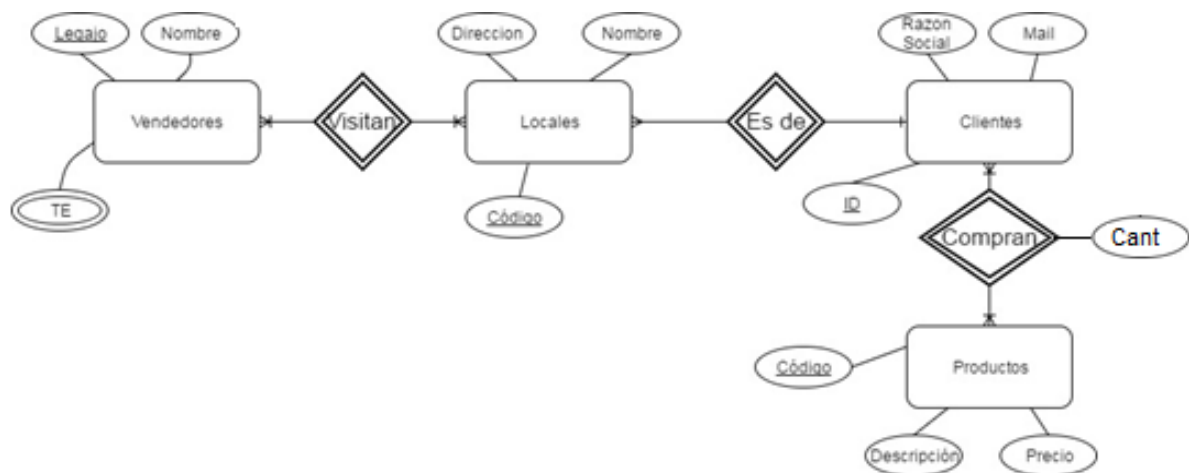
- ✓ Analizar el uso de las sentencias DML: select



2. USO DE SENTENCIAS DML: SELECT, INSERT, UPDATE, DELETE

A continuación vamos a aprender a leer o consultar las tablas que hemos creado suponiendo que ya están llenas de datos, por supuesto.

Tomaremos el diagrama E-R siguiente para interactuar con los objetos derivados del mismo e iremos trabajando en base a ejemplos prácticos para guiar el aprendizaje.



Para consultar los datos de una o varias tablas, se utiliza la sentencia SELECT. La forma más básica de esta sentencia es:

SELECT<lista de atributos>**FROM**<lista de tablas>

WHERE<lista de condiciones>

ORDER BY<lista de atributos><asc, desc>

Si quisiéramos seleccionar todos los atributos, en lugar de listarlos de a uno separados por comas, podemos utilizar el * (asterisco) que simboliza justamente “todos los campos”. Por ejemplo utilizando esta forma del **SELECT** para hacer un listado de las descripciones y precios de los productos ordenados alfabéticamente hay que ejecutar la siguiente sentencia:
SELECT descripcion, precio **FROM** productos
ORDER BY descripcion;

Con esta consulta obtendríamos un listado como el siguiente:

Descripcion	Precio
Azúcar	\$ 15
Harina	\$ 10
Huevo	\$1,70
Leche	\$ 22

Si en lugar de querer todos los productos quisiéramos solamente la lista de aquellos cuyo precio es inferior a \$ 20 y ordenados en forma descendente de acuerdo al precio (es decir los más caros al comienzo de la lista):

SELECT descripcion, precio **FROM** productos
WHERE precio<20
ORDER BY precio desc;

Obtendríamos este listado:

Descripcion	Precio
Azúcar	\$ 15
Harina	\$ 10
Huevo	\$1,70

Vemos que el orden ascendente es el que se toma por defecto y por tanto si no se pone nada en la cláusula **ORDER BY** significa que es ascendente.



Quizás se preguntará: ¿Cómo se hace si necesitamos obtener los nombres de los clientes y los productos que compraron, con sus cantidades?

Pues bien, se puede hacer de dos maneras, en la primera utilizaremos la cláusula **WHERE** para armar la correspondencia entre las t-uplas que participan en la relación (la unión entre la clave primaria de la tabla clientes y la clave primaria de la tabla productos). Para referirse a los atributos o campos de una tabla se utiliza la siguiente notación: tabla.atributo de manera que se puedan diferenciar los nombres de los atributos entre las tablas, sobre todo en los casos en que el nombre es el mismo (por ejemplo el código). Hacemos entonces:

SELECT Clientes.Razon_Social, Productos.descripcion, Productos.precio, Clientes_Productos.cant **FROM** Productos, Clientes, Clientes_Productos **WHERE** Productos.Codigo= Clientes_Productos.CodProducto and Clientes.Codigo= Clientes_Productos.CodCliente;

Se obtendrá una lista como la siguiente:

Razón Social	Descripción	Precio	Cant
El Alfil	Harina	10	31
La Salamandra	Azúcar	15	11
La Salamandra	Huevo	1,70	12
El Alfil	Huevo	1,70	18
La Salamandra	Leche	22	13
El Alfil	Leche	22	15
La Salamandra	Harina	10	8
El Alfil	Azúcar	15	10

Si necesitamos mostrar todos los atributos de una tabla pero no de la otra ponemos tabla.*, por ejemplo cliente.* se usaría para mostrar todos los atributos de la tabla clientes. En cambio si utilizamos * sin especificar, se mostrarán todos los atributos de todas las tablas.

Se pueden usar alias para reemplazar el nombre de las tablas en la consulta y no tener que escribir tanto, por ejemplo podríamos usar c para Clientes, p para Productos y cp para Clientes_Productos. Y quedaría entonces así:

SELECT c.Razon_Social, p.descripcion, p.precio, cp.cant **FROM** Productos p, Clientes c, Clientes_Productos cp **WHERE** p.Codigo= cp.CodProducto and c.Codigo= cp.CodCliente;

Si además quisiéramos calcular el monto gastado incluyendo el I.V.A. habría que hacer este cálculo: precio*cant*1,21 para cada t-upla, llamando a este cálculo con el nombre (o alias) monto, es decir se debería modificar la consulta de la siguiente manera:

SELECT c.Razon_Social, p.descripcion, p.precio, cp.cant, p.precio*cp.cant*1,21 monto **FROM** Productos p, Clientes c, Clientes_Productos cp **WHERE** p.Codigo= cp.CodProducto and c.Codigo= cp.CodCliente;

Obtenemos así el siguiente listado:

Razón Social	Descripción	Precio	Cant	monto
El Alfil	Harina	10	31	310
La Salamandra	Azúcar	15	11	165
La Salamandra	Huevo	1,70	12	19,40

El Alfil	Huevo	1,70	18	27,90
La Salamandra	Leche	22	13	286
El Alfil	Leche	22	15	330
La Salamandra	Harina	10	8	80
El Alfil	Azúcar	15	10	150



Habíamos dicho que había dos formas de hacer consultas que “cruzaban” varias tablas a través de sus relaciones. La primera que es la que vimos recién involucra el uso de la cláusula **WHERE** y la otra forma utiliza una nueva cláusula: el **JOIN**.

El **JOIN** se utiliza para indicar la manera en que se están relacionando las tablas, es decir, con que atributos se está plasmando la relación entre ellas. Se escribe de la siguiente forma:

```
SELECT <lista de atributos> FROM tabla1
JOIN tabla2 ON tabla1.campo1=tabla2.campo2
JOIN tabla3 ON tabla2.campo3=tabla3.campo4
```

Entonces, para escribir la misma consulta que antes pero utilizando el **JOIN** haríamos así:
SELECT c.Razon_Social, p.descripcion, p.precio, cp.cant, p.precio*cp.cant*1,21 Monto **FROM**
 Productos p

JOIN Clientes_Productos cp **ON** p.Codigo= cp.CodProducto

JOIN Clientes c **ON** c.Codigo= cp.CodCliente;

Observe que el **WHERE** no se utiliza a menos que sea para reales condiciones que correspondan a características propias de los datos, por ejemplo si deseamos que solo muestren los datos de los productos para los cuales el precio sea menor a \$ 20:

```
SELECT c.Razon_Social, p.descripcion, p.precio, cp.cant, p.precio*cp.cant*1,21 Monto FROM
Productos p
```

JOIN Clientes_Productos cp **ON** p.Codigo= cp.CodProducto

JOIN Clientes c **ON** c.Codigo= cp.CodCliente

WHERE precio<20;

Cláusula IN:

Ahora veremos, cómo obtenemos los códigos de ciertos productos específicos, por ejemplo Harina, Azúcar y Leche:

```
SELECT código FROM producto WHERE descripción ='Harina' OR descripción ='Azúcar' OR
descripción ='Leche'
```

Pero si tenemos una lista larga de posibilidades, escribir todas estas cláusulas **OR** encadenadas sería muy tedioso, entonces usamos la sentencia **IN** que funciona de manera equivalente:

```
SELECT código FROM producto WHERE descripción IN ('Harina', 'Azúcar', 'Leche')
```

Siempre después de la cláusula IN va una lista de una sola columna o atributo, y siempre antes va un atributo que debe coincidir en su tipo de dato con el tipo de dato de la lista.

Cláusula LIKE:

Otra cláusula muy potente del lenguaje SQL es la que se utiliza para comparaciones con campos de tipo de cadenas de texto. Esta sentencia se podría utilizar por ejemplo para consultar cuáles son los clientes que viven en una calle que contiene el nombre Martín, pero que no se sabe si se ha escrito Martín o Martin (o sea que podría estar sin acento). ¡O sea podríamos tener en esta lista gente que viva en la localidad de San Martí o en la localidad Martín Coronado o en Martínez!

La cláusula de la que estamos hablando es el **LIKE**. Veamos cómo se utiliza en este caso:

```
SELECT * FROM Clientes c
WHERE calle LIKE '%Mart[ií]n%'
```

Analicemos un poco esta comparación para entender mejor como trabaja el **LIKE**:

Primero que nada:

Las comparaciones que trabajan con **LIKE** van todas entre comillas simples

Existen comodines como ser:

%: este comodín representa una cadena de cualquier largo que incluye texto, números y espacios en blanco

_: este comodín representa un solo carácter pero este puede ser una letra o un número

?: este comodín representa un solo carácter de tipo numérico (o sea un dígito)

[]: entre corchetes vamos a poder colocar todos aquellos caracteres o números posibles que pueden ir en un solo lugar. O sea es como si pusiéramos el guion bajo pero le diéramos solo una lista de opciones posibles refinando así los valores que se pueden poner en ese lugar.

Veamos entonces en detalle lo que pasó con nuestra consulta:

Al colocar el % al comienzo y al final estamos representando un texto que no nos preocupa como comienza ni cómo termina, siempre y cuando contenga la palabra que nos interesa. Como no sabíamos si iba a estar escrito o no con acento entonces colocamos entre corchetes las dos opciones de i (o sea i e í).

Otro ejemplo:

Buscar los nombres de las calles que comiencen con N o J, luego viene una vocal y a continuación un texto cualquiera que termina con dos números. Esta condición sería así:

```
SELECT * FROM Clientes c
WHERE calle LIKE '[NJ][aeiou]%%??'
```



ACTIVIDAD 2

Le pedimos realice a continuación la actividad de Autoevaluación que le proponemos en el campus de modo de ir afianzando los conceptos vistos



ABORDEMOS EL LOGRO DEL TERCER OBJETIVO DE LA UNIDAD:

- ✓ Conocer cómo funcionan las consultas



3. USO DE SENTENCIAS DML: USO DE SUBCONSULTAS.

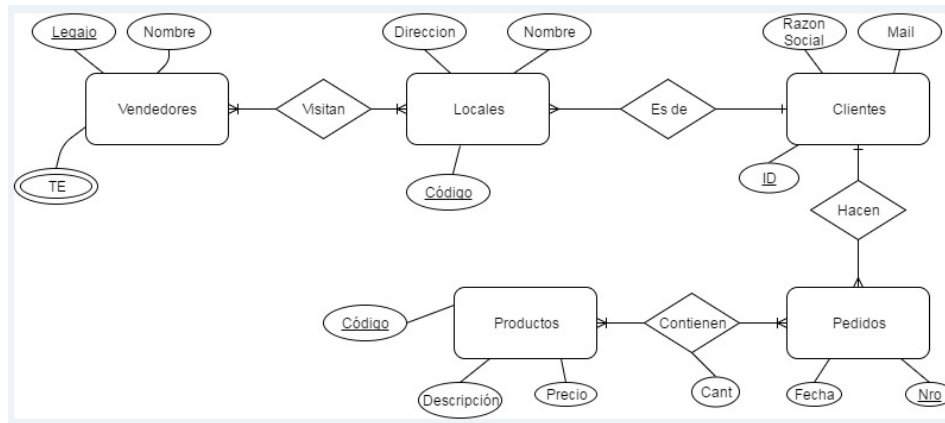
En este apartado avanzaremos un poco más sobre las consultas y veremos algunos trucos más avanzados de lenguaje SQL.

Por ejemplo, si quisiéramos saber cuáles son los clientes que NO compraron ningún producto en un cierto mes o en un rango de tiempo dado, tendríamos que seguir el siguiente razonamiento:

Primero deberíamos ver cuáles son los clientes que efectivamente compraron en el período dado, y a continuación deberíamos pedir los códigos de los clientes que no están en esa lista. Sencillo no?

Como nos iremos dando cuenta durante el estudio de esta unidad, en SQL siempre encontraremos varias formas de hacer consultar los datos y llegar al mismo resultado, por eso, en adelante te vamos a mostrar siempre más de una opción de escribir una consulta.

Veamos concretamente con un ejemplo:
Para el siguiente negocio cuyo diagrama E-R es:



suponiendo que queremos los clientes que no compraron productos en el mes de enero de 2017

Primero sacamos los clientes que compraron en enero de 2017:

```
SELECT codcliente FROM pedidos WHERE month(fecha)=1 and year(fecha)=2017
```

Y ahora necesitaríamos justamente los clientes que no son los de esta lista, por eso pedimos todos los datos de la tabla clientes siempre y cuando el código no esté en la lista de los que hicieron los pedidos del mes de enero de 2017:

```
SELECT * FROM clientes WHERE código not IN (SELECT codcliente FROM pedidos WHERE month(fecha)=1 and year(fecha)=2017)
```

Cláusula EXISTS and NOT EXISTS:

Veamos otra manera de hacer esto mismo pero usando la cláusula **EXISTS**:

El **EXISTS** tiene una consulta interna y una externa, que están unidas por una condición **WHERE** de manera que, si la consulta interna devuelve datos para una determinada T-upla, entonces la consulta externa devuelve esa T-upla, y si la consulta interna no devuelve datos, entonces no se devuelve la T-upla.

Parece complicado pero veámoslo con el ejemplo anterior

Condición de unión entre la consulta interna y externa

```
SELECT * FROM clients c WHERE EXISTS (SELECT * FROM pedidos p WHERE month(fecha)=1 and year(fecha)=2017 and p.codcliente=c.codigo)
```

Consulta Externa

Consulta Interna

Y ahora con los datos:
Si la tabla Pedidos y la tabla Clientes tienes las siguientes T-uplas:

Nro	Fecha	CodCliente
1	21/10/2016	14
2	13/11/2016	24
3	15/12/2016	17
4	3/1/2017	13
5	10/1/2017	22
6	28/1/2017	14
7	3/3/2017	24
8	9/4/2017	19

	Razon Social	Mail
14	La Salamandra	Salamandra@fuego.com
13	El Alfil	Alfil@alfil.com
17	Don Luis	DonLuis@donluis.com
22	Los Patitos	Info@patitos.com
24	Ramos Generales	rgrales@genstore.com
19	El Arca	info@elarca.com

Recordemos la consulta:

```
SELECT * FROM clients c WHERE EXISTS (SELECT * FROM pedidos p WHERE month(fecha)=1 and year(fecha)=2017 and p.codcliente=c.codigo)
```

Las T-uplas correspondientes al resultado de la consulta interna son:

ID	Fecha	CodCLiente
4	3/1/2017	13
5	10/1/2017	22
6	28/1/2017	14

Para estas T-uplas, la tabla de la relación Pedidos_Productos corresponde a:
La tabla Pedidos_Productos esta formada por las T-uplas:

CodPedido	CodProducto	Cant
4	2	15
4	3	25
5	2	21
5	4	33
5	5	29
6	1	13
6	2	12

Luego, si observamos la relación entre ambas consultas:
Lo que va a estar haciendo es tomar los clientes de la lista clientes que además estén en dicho conjunto de T-uplas, es decir el 13, el 22 y el 14.

Por lo tanto al final, como en realidad en la consulta externa estamos pidiendo todos los datos de los clientes que reúnan esa condición, lo que tendremos son las T-uplas de la tabla Clientes para los ID 13,22 y 14. Es decir:

ID	Razon Social	Mail
14	La Salamandra	Salamandra@fuego.com
13	El Alfil	Alfil@alfil.com
22	Los Patitos	Info@patitos.com



Para repasar todos los pasos seguidos utilizando el entorno visual MySQL Workbench, sería ahora un buen momento para que vea el video titulado “Subconsultas” que se encuentra en el campus.



ACTIVIDAD 3

Le pedimos que a continuación realice la autoevaluación que le proponemos en el campus a fin de ir afianzando conceptos vistos hasta aquí



ABORDEMOS EL LOGRO DEL CUARTO OBJETIVO DE LA UNIDAD:

- ✓ Conocer cómo funcionan las funciones agregadas



4. USO DE FUNCIONES AGREGADAS

En este apartado vamos a ver una de las características que hacen del lenguaje SQL uno de los más usados y potentes cuando se trata de armar reportes. Aun hoy en los tiempos de Big Data, SQL sigue siendo popular por la funcionalidad asociada a la agregación de datos. Es decir, a la posibilidad de calcular sumas, promedios, entre otros, a partir de conjuntos de T-uplas agrupando por diferentes atributos que suelen denominarse **dimensiones**. Las funciones de agregación más comunes disponibles en el lenguaje, y aquellas en las que nos enfocaremos para los ejemplos, son: SUM, AVG, MAX, MIN, COUNT .

La sintaxis del uso de las funciones agregadas es como sigue:

SELECT <lista de campos>, función agregada **FROM** <tabla1 **JOIN** tabla2 **ON**....>

GROUP BY <lista de campos>

[**HAVING** función agregada <condición>]

Obviamente se utilizan conjuntamente con el **SELECT** ya que siempre van asociadas a una consulta. Además conceptualmente lo que hacen es reunir un conjunto de T-uplas de

manera de juntar los datos para poder llevar a cabo la operación en cuestión (suma, promedio, cuenta, etc), por lo tanto van a **agrupar** T-uplas, de ahí la necesidad de la cláusula **GROUP BY**.

Debés tener en cuenta que la <lista de campos> en el **GROUP BY** y en el **SELECT** es la misma. Si no hay una lista de campos, quiere decir que vamos a obtener una suma total, por lo tanto la cláusula **GROUP BY** tampoco es necesaria.

Veamos ahora un ejemplo para entender cómo funciona la suma, y luego lo extenderemos al resto de las funciones agregadas citadas.

Si queremos saber cuánto se vendió en el mes de enero, o sea de las 3 ventas que vimos recién, quisiéramos calcular su suma total, multiplicando el precio por la cantidad y sumando obtendríamos un solo número \$1936.5 calculado así:

$15*10+25*1.7+21*10+33*22+29*17+13*15+12*10$



Seguramente se preguntará ahora ¿Cómo haríamos este mismo cálculo con el lenguaje SQL? Y además ¿Cuál es la potencia de SQL en este tipo de cálculo?

Pues bien, para realizar una consulta que calcule esto mismo usaremos la consulta interna del ejemplo anterior:

```
SELECT SUM(cant*pr.precio) FROM pedidos_productos pp
JOIN productos pr ON pp.codproducto=pr.codigo
JOIN pedidos p ON p.nro=pp.codpedido
WHERE month(p.fecha)=1 and year(p.fecha)=2017
```

Observe que tenemos que usar la tabla pedidos para poder usar la condición del mes= enero y el año=2017 con la fecha, la tabla productos porque necesitamos los precios y la tabla pedidos_productos porque necesitamos la cantidad comprada de cada producto por cada pedido. Por eso necesitamos hacer esos tres join.

Además...

¡Claro que se habrá dado cuenta que la potencia está en que ahora podremos calcular una cuenta similar no solo para el mes de enero sino para todos los meses y podremos entonces saber la venta mensual, y a partir de allí la venta anual!

De manera similar, si quisiéramos saber cuántas unidades se vendieron de cada producto (no importa si son kg, litros o unidades), podríamos hacer así:

De manera similar, si quisiéramos saber cuántas unidades se vendieron de cada producto (no importa si son kg, litros o unidades), podríamos hacer así:

```
SELECT SUM(cant) FROM pedidos_productos pp
JOIN productos pr ON pp.codproducto=pr.codigo
JOIN pedidos p ON p.nro=pp.codpedido
WHERE month(p.fecha)=1 and year(p.fecha)=2017
```

Y ahora, vamos a poner el mes para obtener la suma de las ventas de cada uno de los meses del año 2017, así veremos cómo trabaja el **GROUP BY** :

```
SELECT month(p.fecha) as Mes, SUM(cant) as Cantidad FROM pedidos_productos pp
JOIN productos pr ON pp.codproducto=pr.codigo
JOIN pedidos p ON p.nro=pp.codpedido
WHERE year(p.fecha)=2017
GROUP BY month(p.fecha)
```

Esta consulta nos retornará las siguientes T-uplas:

Mes	Catidad
1	148
3	174
4	130

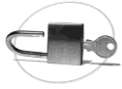
Si hubiéramos querido saber los pedidos cuyo total fuera superior a \$ 1000 hubiéramos tenido que hacer lo siguiente:

```
SELECT p.Nro, SUM(cant*precio) as total FROM pedidos_productos pp
JOIN productos pr ON pp.codproducto=pr.codigo
JOIN pedidos p ON p.nro=pp.codpedido
GROUP BY p.nro
HAVING SUM(cant*precio)>1000
```



ACTIVIDAD 4

Le pedimos que a continuación realice la autoevaluación que le proponemos en el campus a fin de ir afinizando conceptos vistos hasta aquí



ABORDEMOS EL LOGRO DEL QUINTO OBJETIVO DE LA UNIDAD:

- ✓ Conocer cómo funcionan las sentencias para modificar datos



5. USO DE SENTENCIAS DML PARA MODIFICAR DATOS: INSERT, UPDATE, DELETE.

Como habrá observado, hasta ahora estuvimos siempre trabajando con tablas asumiendo que tenían datos.



Se preguntará entonces, ¿Qué pasa cuando necesitamos poner una T-upla o más aún, modificar un atributo de la misma?

Se habrá imaginado que existen sentencias DML especiales para realizar estas actividades dentro del lenguaje SQL. Ellas son:

Para agregar T-upla de datos: **INSERT**

Para modificar atributos de una o varias T-uplas: **UPDATE**

Para borrar T-uplas completas de una tabla: **DELETE**

INSERT

La sintaxis más simple del INSERT es así:

INSERT INTO tabla (<lista de atributos>)

VALUES (lista de valores)

La lista de atributos es opcional, pero si no se pone entonces el DBMS espera una lista de valores coherente con todos los atributos de la tabla. Veamos un ejemplo para entender bien la sintaxis.

Supongamos nuestra tabla de Productos del diagrama E-R que veníamos utilizando en los apartados anteriores:

Productos: {Código, Precio, Descripción}

Para insertar un nuevo producto utilizando la sintaxis indicada haríamos así:

Si ponemos todos los valores de cada uno de los atributos al momento de insertar no necesitamos poner la lista, es decir las tres sentencias siguientes son equivalentes:

INSERT INTO productos (codigo, precio, descripcion)

VALUES (26, 14, 'Fecula de maíz')

INSERT INTO productos (precio, codigo, descripcion)

VALUES (14, 26, 'Fecula de maíz')

O por ejemplo si hubiéramos hecho:

INSERT INTO productos

VALUES (14, 26, 'Fecula de maíz')

Habría tomado 26 como el precio, ya que está en el lugar que corresponde a ese atributo en la definición de la tabla.

O si usáramos:

INSERT INTO productos (precio, codigo, descripcion)

VALUES (26, 14, 'Fecula de maíz')

Habría tomado 26 como el precio y no como el código, ya que está en el lugar del precio de acuerdo a la lista de atributos utilizada en la sentencia.

Es decir, si se coloca la lista de atributos, la lista de valores sigue esa secuencia, si no se coloca, el DBMS asume que la lista de valores está ordenada de acuerdo a la definición de la tabla.

UPDATE

Ahora veamos la sentencia **UPDATE** en su forma más simple:

UPDATE tabla

SET campo1=valor1,

Campo2= valor2,

....

campoN= valorN

WHERE <lista de condiciones>

CUIDADO: ¡Si no hay cláusula **WHERE** lo que ocurrirá es que se actualizarán todas las T-uplas de la tabla!

Esto es un error muy común que cometen algunos usuarios avanzados, ya hablaremos sobre ese tema en la unidad 5.

Veamos algunos ejemplos de aplicación:

Si queremos modificar el precio del producto de código 14 que acabamos de insertar en el apartado anterior, quizás en lugar de 26 queremos que sea \$28 podemos hacer lo siguiente:



ACTIVIDAD 5

Le pedimos que a continuación realice la autoevaluación que le proponemos en el campus a fin de ir afinizando conceptos vistos hasta aquí

UPDATE productos
SET precio= 28
WHERE codigo=14;

O quizás queríamos incrementar su precio en un 15%, en cuyo caso haríamos así:

UPDATE productos
SET precio= precio * 1.15
WHERE codigo=14;

Si lo queríamos disminuir en un 15%, hubiéramos hecho así:

UPDATE productos
SET precio= precio * 0.85
WHERE codigo=14;

PERO, si lo que necesitábamos hacer era incrementar los precios de TODOS los productos en un 10%, la sentencia no utiliza la cláusula **WHERE** como habíamos indicado anteriormente.

UPDATE productos
SET precio= precio * 1.1;

DELETE

Por último, para eliminar registros de una tabla utilizamos ésta sentencia. Su forma más simple es:

DELETE FROM tabla
WHERE <lista de condiciones>;

Acá hacemos la misma salvedad que en el caso del **UPDATE**, si no se usa la cláusula **WHERE** lo que ocurrirá es que se eliminarán TODAS las T-uplas de la tabla y la misma quedará vacía, lo que no es la intención habitual.

Usemos un ejemplo para ilustrar cómo borraríamos el producto que acabamos de ingresar cuando vimos la sentencia **INSERT**:

```
DELETE FROM productos  
WHERE codigo=14;
```

Y si hacemos lo siguiente:
DELETE FROM productos;

¡Nos quedamos sin productos!



ABORDEMOS EL LOGRO DEL SEXTO OBJETIVO DE LA UNIDAD:

- ✓ Conocer cómo interactuar con NULLs



6. INTERACCIÓN CON NULLS

Es posible que al ingresar los datos de una tabla de la base de datos, no conozcamos el valor de alguno de sus atributos, entonces lo que debemos hacer es no ingresar el valor correspondiente. Por ejemplo si tenemos que insertar un nuevo cliente y no conocemos su email deberíamos hacer lo siguiente:

```
INSERT INTO clientes (ID, razon_social)  
VALUES (32,'El dulce cañon')
```

Y NO lo siguiente:

```
INSERT INTO clientes
VALUES (32,'El dulce cañon', '')
0
INSERT INTO clientes (ID, razon_social,email)
VALUES (32,'El dulce cañon', '')
```

Podrá ver que en el caso correcto (el primer caso), no se está listando el atributo que se desconoce en la lista de campos del insert y por lo tanto no se coloca en valor en la cláusula **VALUES**.



Se preguntará entonces qué sucede con el valor del email cuando se ingresa el registro en la tabla, ¿Qué valor queda ingresado?

¡Justamente es de lo que estamos hablando! El valor ingresado en ese atributo para este caso es un valor especial denominado NULL. Lo habíamos utilizado cuando definimos la sentencia CREATE TABLE y dijimos que se utilizaba NOT NULL cuando se trataba de atributos para los que no era obligatorio colocar el valor en la tabla.

¡Pues de eso se trata! Como no sabemos qué valor poner, no ponemos nada. Eso se traduce dentro de la base de datos en un valor NULL.

El NULL y el blanco no son lo mismo, el blanco es una cadena de caracteres vacía, pero es un valor de todas maneras.



¿Pero cuál es la diferencia entre la cadena vacía y un NULL? ¿No son lo mismo?

No son lo mismo para el DBMS, porque el NULL significa que el atributo en cuestión es desconocido. El NULL no es un valor, sino un estado del atributo. El DBMS maneja los atributos cuyo estado es NULL de una manera óptima, los puede clasificar y encontrar más rápido y además, para nosotros que interactuamos con la base de datos es mucho mejor para encontrar esos datos y poder manejarlos posteriormente.

Es siempre mejor usar NULLs cuando desconocemos el valor a ingresar que ingresar cualquier otra cosa, o '**Valor Desconocido**' o incluso blancos.

¿Por qué? ¿Si es un número, que valor ingresaríamos para simbolizar el valor desconocido?

¿0? ¡No es lo mismo un precio=0 que un precio desconocido!

¡Otra muy buena razón es que el 0 ocupa lugar y el NULL no!



Ahora seguro querrá saber ¿Cómo encontramos estos NULLs si los queremos buscar?

¡Lo que haremos es simplemente consultar por ellos! Y se hace de la siguiente manera:

```
SELECT * FROM tabla  
WHERE campo IS NULL
```

No se debe usar el = ya que como dijimos anteriormente, el NULL es un valor desconocido y como tal, no se puede saber si es igual o diferente a cualquier otro valor. De hecho las dos siguientes condiciones resultan falsas ya que como es un valor desconocido sería imposible definir si es igual a otro valor desconocido (pueden ser desconocidos pero diferentes entre sí). Entonces:

```
NULL=NULL  
NULL<>NULL
```

Por ejemplo, si queremos saber cuales son los productos cuya descripción es desconocida deberíamos hacer la siguiente consulta:

```
SELECT * FROM productos  
WHERE descripcion IS NULL
```

Con lo cual no es correcto escribir la misma comparación de la siguiente forma:

```
SELECT * FROM productos  
WHERE descripcion = NULL
```

De la misma manera, para preguntar si el precio es desconocido se escribe así:

```
SELECT * FROM productos  
WHERE precio IS NULL
```



ABORDEMOS EL LOGRO DEL ÚLTIMO OBJETIVO DE LA UNIDAD:

- ✓ Conocer cómo funcionan las transacciones.



7. TRANSACCIONES

Ya vimos todas las sentencias del lenguaje SQL que se utilizan para interactuar con la base de datos y obtener información a partir de sus datos.

Supongamos que nuestra base de datos se utiliza para gestionar las cuentas de un banco y que Juan Perez, cuyo código de CBU es el 23 tiene que hacer una transferencia de dinero a Pedro González cuyo código de CBU es 44. El monto que va a transferir es de \$550.

La transferencia involucra que Juan va a retirar los \$550 de su cuenta y luego va a ingresar a la cuenta de Pedro es misma suma de dinero. Si lo pensamos en operaciones DML como las que vimos en los apartados anteriores, lo podríamos realizar con dos updates, uno que incremente el saldo en 550 (en la cuenta de Pedro) y otro que lo disminuya en 550 (en la cuenta de Juan).

Pero hay que pensar que estamos interactuando con la base de datos desde una aplicación, por ejemplo un sitio parecido a pagomiscuentas.com o el sitio de uno de los bancos. Entonces Juan coloca los datos del CBU de la cuenta de Pedro y presiona el botón "Transferir".



Se preguntará ahora: ¿Qué pasaría si se corta la conexión de internet de Juan en medio que está realizando esta operación?

¡Este es un detalle muy importante! Debemos tener en cuenta a la hora de interactuar con los datos y las tablas es el hecho de asegurarnos que las dos operaciones ocurran al mismo tiempo, y si por algún motivo una de ellas se cancela, también debe cancelarse la otra.

Una transacción es una unidad atómica de operación que debe ser realizada completamente o cancelada en caso de que suceda alguna falla.

En el caso del ejemplo de la transferencia bancaria es muy importante que se aplique este comportamiento ya que si falla el depósito en la cuenta de Pedro pero no la extracción de la cuenta de Juan, ¿Dónde quedarían los \$550 de Juan?

Entonces consideraremos esta operación como si fuera una transacción, es decir si falla una de ellas, se cancela todo y los saldos de ambos serían los mismos que antes de comenzar la transferencia.

Propiedades ACID de las transacciones

Las transacciones tienen que cumplir cuatro propiedades:

- ❖ **Atomicidad:** se realizan todas las operaciones involucradas con éxito o no se realiza ninguna.
- ❖ **Consistencia:** la base de datos pasa de un estado consistente a otro.
- ❖ **Aislamiento:** cada transacción es aislada, es decir que mientras está ocurriendo ninguna otra transacción puede acceder a los datos que están siendo modificados por la misma.
- ❖ **Durabilidad:** los cambios ocurridos en la base de datos por causa de la transacción son persistentes, es decir que no se pueden perder por causa de una falla.

Por este motivo se dice que las transacciones tienen propiedades ACID (por sus siglas en inglés). En el lenguaje SQL, para marcar el comienzo de una transacción, se utiliza la sentencia **START TRANSACTION** y para marcar el fin de la transacción se utiliza **COMMIT**. Si en lugar de cerrar la operación deseamos cancelar los cambios tenemos que ejecutar la sentencia **ROLLBACK**.

Entonces, volviendo al ejemplo de la transferencia de Juan a Pedro, se haría de esta manera:

START TRANSACTION

UPDATE movimientos

SET monto=monto - 550

WHERE CBU= (select CBU from cuentas where ID=23)

UPDATE movimientos

SET monto=monto +550

WHERE CBU= (**SELECT** CBU **FROM** cuentas **WHERE** ID=44)

COMMIT

El DBMS implementa mecanismos que refuerzan el comportamiento de las transacciones y aseguran que ante una falla los estados de todas las tablas involucradas vuelvan a ser los mismos que antes de que comenzara la transacción. Estos son los llamados **mecanismos de**

recuperación, ellos utilizan puntos de control o marcas de tiempo que les permiten poder volver a esos estados anteriores al comienzo de la transacción. Veremos estos conceptos en la unidad 4.

Muchas veces es útil trabajar con variables cuando usamos transacciones. Los nombres de las variables en lenguaje SQL se anteponen con el símbolo @ y a continuación una sucesión de caracteres, por ejemplo @nom21. Se deben declarar previamente a su utilización indicando su tipo. Vemos a continuación un ejemplo muy común del uso de una variable:

```
set @maxmonto=(select max(monto) from movimientos)
select @maxmonto
```

Con esta serie de comandos se ha creado la variable @maxmonto, y se le ha asignado el valor del máximo monto de la tabla movimientos, para luego mostrarlo en la siguiente sentencia.



ACTIVIDAD 6

Le pedimos que a continuación realice la Actividad de autoevaluación que le proponemos en el campus a modo de ir afianzando los conceptos vistos hasta aquí.



Si la Actividades fueron verificadas por su tutor usted habrá logrado comprender las principales características del lenguaje SQL para consultas.

¡FELICITACIONES!

Ha logrado alcanzar los objetivos propuestos para esta Unidad.



SÍNTESIS DE LA UNIDAD

Hagamos un resumen de todo lo visto en esta Unidad.

En esta Unidad vimos los conceptos básicos que usted debe conocer para poder comenzar a trabajar con consultas en lenguaje SQL para poder obtener información a partir de sus bases de datos.

El LENGUAJE SQL se utiliza para resolver CONSULTAS en una BASE DE DATOS. El mismo contiene SENTENCIAS DML, SENTENCIAS SDL y SENTENCIAS DDL,

Las SENTENCIAS DDL son CREATE, ALTER y DROP que se utilizan también para generar SENTENCIAS SDL.

Las SENTENCIAS DML son SELECT, INSERT, UPDATE y DELETE.

El SELECT puede tener OPERACIONES DE AGREGACIÓN que involucran SUM, COUNT, AVG, MAX y MIN.

Las operaciones de SELECT pueden estar formando parte de CONSULTAS o de SUBCONSULTAS.

